

TRAFFIC LIGHT CONTROLLER USING VERILOG HDL

RTL Design and Functional Verification of a Moore Finite State Machine

PROJECT REPORT

SUBMITTED BY

NAME: AMAN MISHRA

SEMESTER: 5TH

YEAR : 3RD

**DEPARTMENT: ELECTRONICS AND COMMUNICATION
ENGINEERING**

TRAFFIC LIGHT CONTROLLER USING VERILOG HDL

RTL Design and Functional Verification of a Moore Finite State Machine

TABLE OF CONTENTS

SECTION	TITLE	PAGE
1	System Overview	2
2	State Encoding Table	2
3	Block Diagram	3
4	State Transition Diagram	4
5	State Transition Table	5
6	Moore Output Table	5
7	Next-State & Output Boolean Expressions	5
8	State Transition Annotations	6
9	Project Execution in Xilinx Vivado	6
9.1	Prerequisites	6
9.2	Step 1: Create a New Project	6
9.3	Step 2: Add Design Source	7
9.4	Step 3: Add Testbench	7
9.5	Step 4: Run Behavioral Simulation	8
10	Simulation Results	8
10.1	Behavioral Simulation	8
10.2	Post-Synthesis Functional Simulation	9
10.3	RTL Schematic	9
10.4	Synthesized Schematic	10
10.5	State Transition Verification	10
10.6	Simulation Result Analysis	11
11	Future Improvements	12
12	Conclusion	12
13	References	12

LIST OF TABLES

Table 1	State Encoding Table	2
Table 2	Present State / Input / Next State (Moore FSM)	5
Table 3	Moore Output Table	5
Table 4	Minimised Boolean Expressions	6

LIST OF FIGURES

Figure 1.....	RTL Block Diagram (Moore FSM)	3
Figure 2.....	State Transition Diagram	4

TRAFFIC LIGHT CONTROLLER

Moore Finite State Machine — State Transition Tables & Diagrams

1. SYSTEM OVERVIEW

This document presents the complete state transition specification for a two-road Traffic Light Controller modelled as a Moore Finite State Machine (FSM). In a Moore FSM, all outputs depend exclusively on the present state, independent of the current input values. The design targets synthesis in Verilog HDL and is suitable for FPGA, ASIC, or VLSI implementation.

- Architecture : Moore FSM
- State bits : 2 flip-flops (Q1, Q0)
- Input signals : Ta (Traffic Sensor, Road A) | Tb (Traffic Sensor, Road B)
- Output signals : La (Signal Head, Road A) | Lb (Signal Head, Road B)
- Total states : 4 (S0 – S3)

2. STATE ENCODING TABLE

Table 1 defines the 2-bit binary encoding assigned to each named state and its corresponding signal output combination.

State	Encoding (Q1Q0)	Description
S0	0 0	Road A: GREEN Road B: RED
S1	0 1	Road A: YELLOW Road B: RED
S2	1 0	Road A: RED Road B: GREEN
S3	1 1	Road A: RED Road B: YELLOW

Table 1 — State Encoding (Q1 = MSB, Q0 = LSB)

3. BLOCK DIAGRAM

Figure 1 illustrates the RTL block architecture of the Moore FSM. The design separates into three functional units: the Next-State Combinational Logic (driven by present state and inputs), the State Register (two D flip-flops clocked on the rising edge), and the Output Combinational Logic (driven by present state only, confirming Moore behaviour).

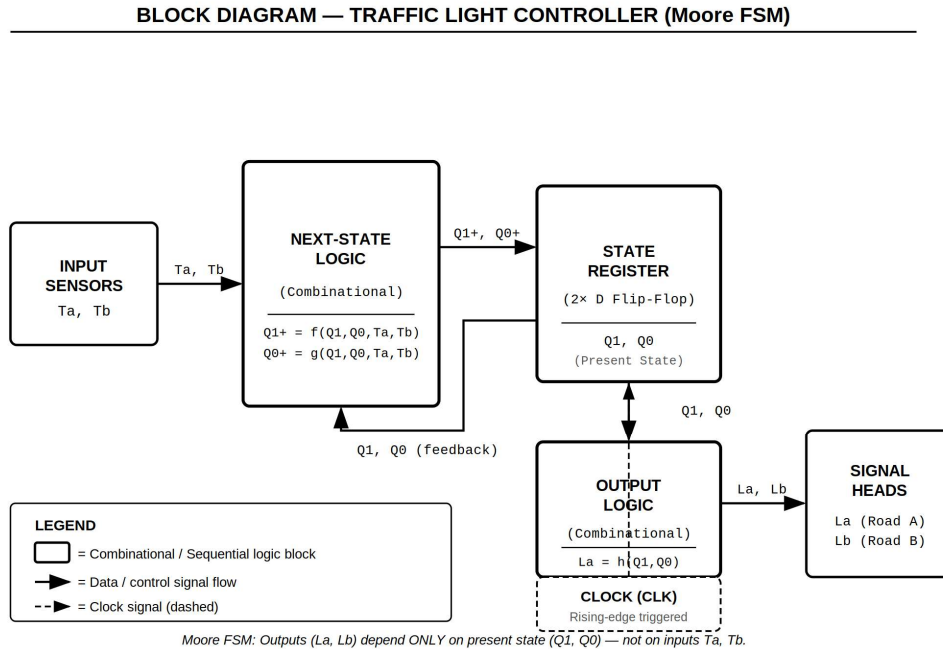
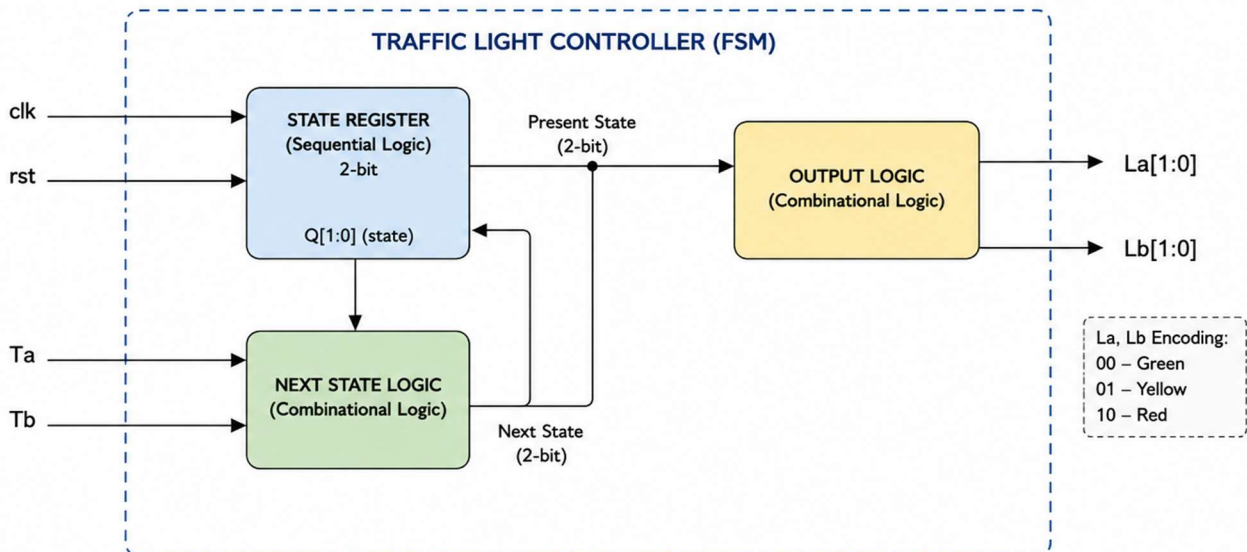


Figure 1 — RTL Block Diagram (Moore FSM: outputs depend only on present state Q1, Q0)



4. STATE TRANSITION DIAGRAM

Figure 2 is the bubble-and-arc state transition diagram. Each circle represents one FSM state with its 2-bit encoding and Moore outputs. Directed arcs carry the input condition that triggers that transition. The self-loops on S0 (Ta=1) and S2 (Tb=1) keep the respective roads green while traffic is detected. Unconditional arcs on S1 and S3 implement fixed yellow-phase interlocks.

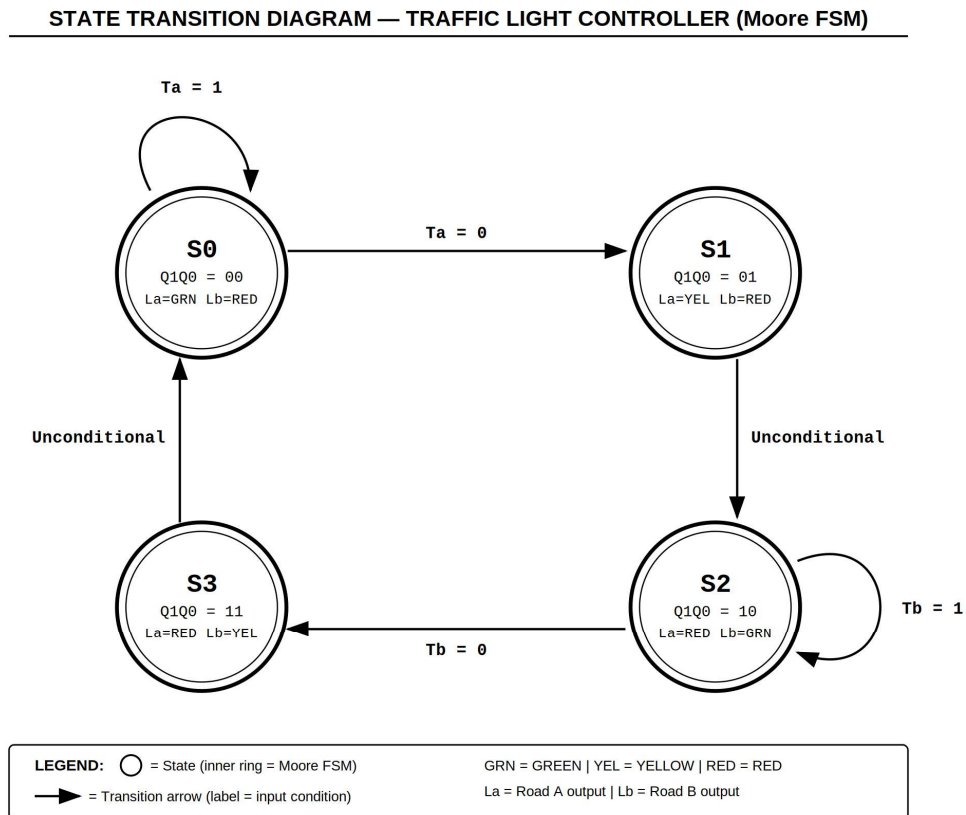


Figure 2 — State Transition Diagram (bubble = state, arc label = input condition)

5. STATE TRANSITION TABLE

Table 2 is the primary FSM specification. It lists every valid combination of present state and active input, together with the resulting next state. The dash symbol (—) denotes a don't-care condition on that input. Next-state bits Q1+ and Q0+ correspond to the encoded present-state values after one clock edge.

Present State	Q1	Q0	Ta	Tb	Next State	Q1+	Q0+
S0	0	0	1	—	S0	0	0
S0	0	0	0	—	S1	0	1
S1	0	1	—	—	S2	1	0
S2	1	0	—	1	S2	1	0
S2	1	0	—	0	S3	1	1
S3	1	1	—	—	S0	0	0

Table 2 — Present State / Input / Next State (Moore FSM)

6. MOORE OUTPUT TABLE

Table 3 lists the traffic signal outputs for each state. Because this is a Moore machine, La and Lb depend solely on the current state encoding (Q1, Q0) and do not change during a clock cycle based on input transitions.

State	Q1	Q0	La (Road A)	Lb (Road B)	Description
S0	0	0	GREEN	RED	Road A Moving
S1	0	1	YELLOW	RED	Road A Warning
S2	1	0	RED	GREEN	Road B Moving
S3	1	1	RED	YELLOW	Road B Warning

Table 3 — Moore Output Table (outputs depend only on present state)

7. NEXT-STATE & OUTPUT BOOLEAN EXPRESSIONS

The combinational logic equations below are derived directly from the state transition table using Boolean minimisation (Karnaugh map). These expressions directly target two-level logic synthesis.

Signal	Boolean Expression
Q1+	$Q1+ = Q1' \cdot Q0 + Q1 \cdot Q0'$
Q0+	$Q0+ = Q1' \cdot Q0' \cdot Ta' + Q1 \cdot Q0' \cdot Tb'$
La	$La[GREEN] = Q1' \cdot Q0' \quad \quad La[YELLOW] = Q1' \cdot Q0$
Lb	$Lb[GREEN] = Q1 \cdot Q0' \quad \quad Lb[YELLOW] = Q1 \cdot Q0$

8. STATE TRANSITION ANNOTATIONS

S0 → S0 (Ta = 1) : Road A traffic detected. Controller holds Road A green to allow continued flow.

S0 → S1 (Ta = 0) : No traffic on Road A. Advances to yellow warning phase.

S1 → S2 (unconditional) : Yellow phase fixed interlock. Unconditionally advances to Road B green after one clock.

S2 → S2 (Tb = 1) : Road B traffic detected. Controller holds Road B green symmetrically.

S2 → S3 (Tb = 0) : No traffic on Road B. Advances to Road B yellow warning phase.

S3 → S0 (unconditional) : Road B yellow expires. Unconditionally returns to S0 completing one full cycle.

9. PROJECT EXECUTION IN XILINX VIVADO

Prerequisites

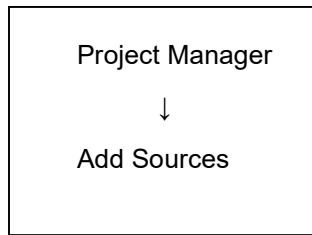
- Xilinx Vivado installed
- Verilog source file (traffic_light_controller.v)
- Testbench file (traffic_light_tb.v)

Step 1: Create a New Project

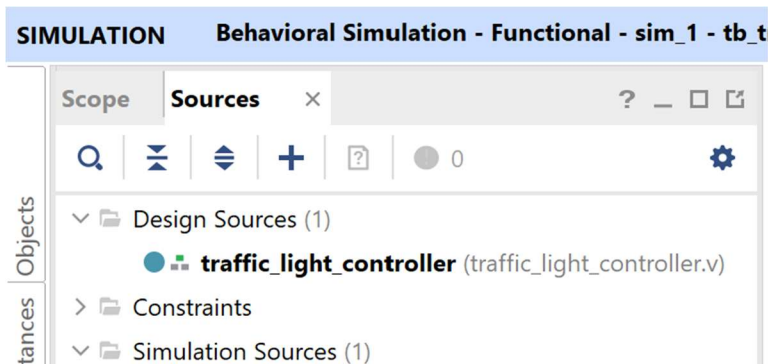
1. Open Vivado.
2. Click **Create New Project**.
3. Enter a project name.
4. Choose a project location.
5. Select **RTL Project**.
6. Check **Do not specify sources at this time** (optional).
7. Click **Finish**.

Step 2: Add Design Source

1. In **Flow Navigator**:



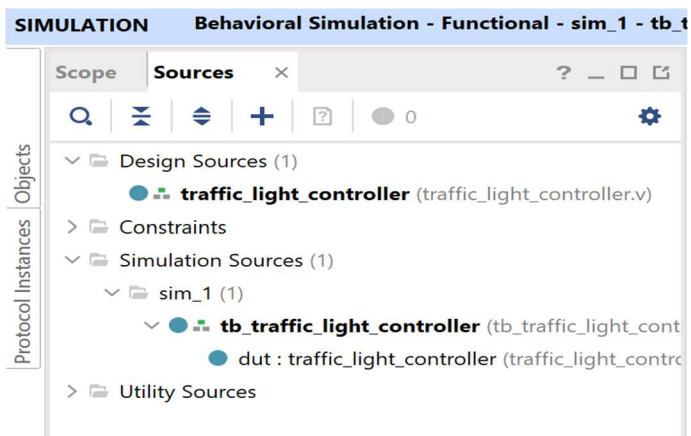
2. Select **Add or Create Design Sources**.



3. Add : traffic_light_controller.v
4. Finish

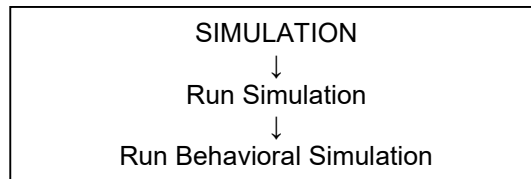
Step 3: Add Testbench1

1. Again select: Add sources
2. Choose : Add or Create Simulation Sources



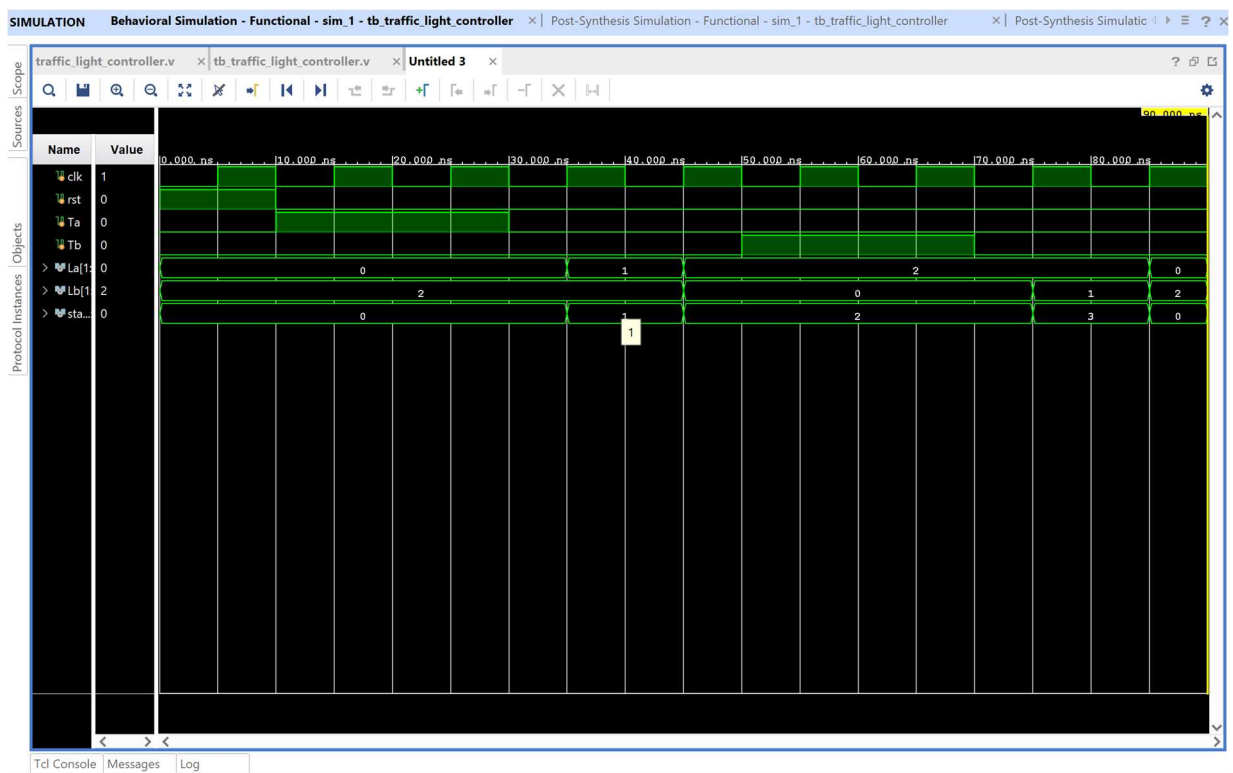
3. Add: traffic_light_tb.v
4. Finish

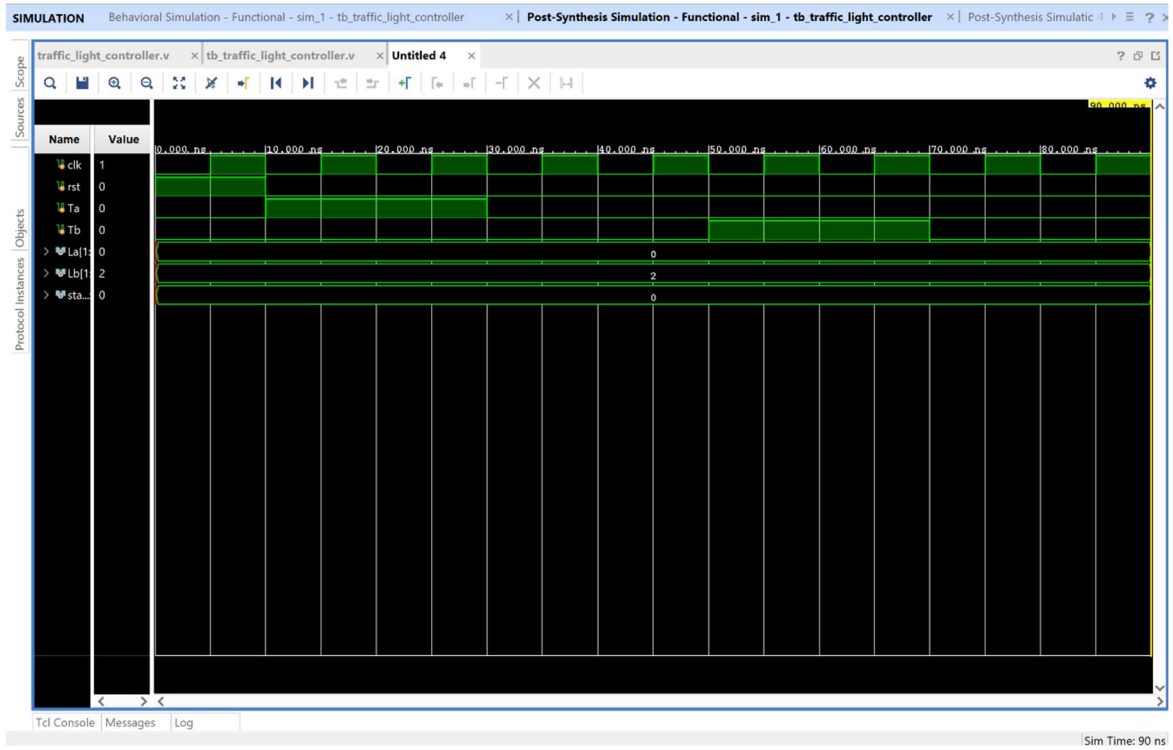
Step 4: Run Behavioral Simulation



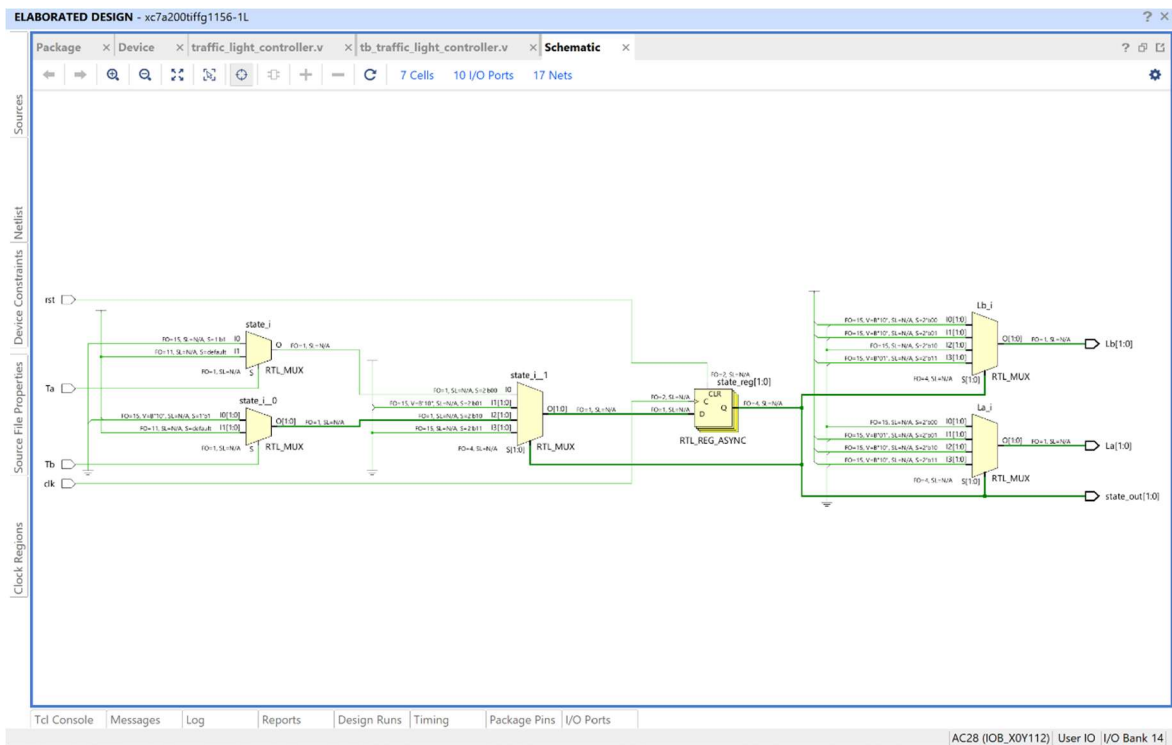
10. SIMULATION RESULTS

- Waveform screenshots





- Schematic





```
# run 1000ns
Time=0 clk=0 rst=1 Ta=0 Tb=0 state_out =00 La=00 Lb=10
Time=5000 clk=1 rst=1 Ta=0 Tb=0 state_out =00 La=00 Lb=10
Time=10000 clk=0 rst=0 Ta=1 Tb=0 state_out =00 La=00 Lb=10
Time=15000 clk=1 rst=0 Ta=1 Tb=0 state_out =00 La=00 Lb=10
Time=20000 clk=0 rst=0 Ta=1 Tb=0 state_out =00 La=00 Lb=10
Time=25000 clk=1 rst=0 Ta=1 Tb=0 state_out =00 La=00 Lb=10
Time=30000 clk=0 rst=0 Ta=0 Tb=0 state_out =00 La=00 Lb=10
Time=35000 clk=1 rst=0 Ta=0 Tb=0 state_out =01 La=01 Lb=10
Time=40000 clk=0 rst=0 Ta=0 Tb=0 state_out =01 La=01 Lb=10
Time=45000 clk=1 rst=0 Ta=0 Tb=0 state_out =10 La=10 Lb=00
Time=50000 clk=0 rst=0 Ta=0 Tb=1 state_out =10 La=10 Lb=00
Time=55000 clk=1 rst=0 Ta=0 Tb=1 state_out =10 La=10 Lb=00
Time=60000 clk=0 rst=0 Ta=0 Tb=1 state_out =10 La=10 Lb=00
Time=65000 clk=1 rst=0 Ta=0 Tb=1 state_out =10 La=10 Lb=00
Time=70000 clk=0 rst=0 Ta=0 Tb=0 state_out =10 La=10 Lb=00
Time=75000 clk=1 rst=0 Ta=0 Tb=0 state_out =11 La=10 Lb=01
Time=80000 clk=0 rst=0 Ta=0 Tb=0 state_out =11 La=10 Lb=01
Time=85000 clk=1 rst=0 Ta=0 Tb=0 state_out =00 La=00 Lb=10
```

Simulation Result Analysis

The simulation results confirm that the Traffic Light Controller FSM operates according to the intended design specifications. The observed state transitions and output responses match the expected behavior defined in the state transition diagram and state transition table.

Verification of State Transitions

The FSM was initialized to **State S0** through the asynchronous reset signal. The following sequence was observed during simulation:

$S0 \rightarrow S0 \rightarrow S0 \rightarrow S1 \rightarrow S2 \rightarrow S2 \rightarrow S2 \rightarrow S3 \rightarrow S0$
--

The behavior of each state was verified as follows:

- **S0 (Road A Green, Road B Red):**
 - When $T_a = 1$, the FSM remained in S0.
 - When $T_a = 0$, the FSM transitioned to S1.
- **S1 (Road A Yellow, Road B Red):**
 - The FSM automatically transitioned to S2.
- **S2 (Road A Red, Road B Green):**
 - When $T_b = 1$, the FSM remained in S2.
 - When $T_b = 0$, the FSM transitioned to S3.
- **S3 (Road A Red, Road B Yellow):**
 - The FSM automatically transitioned back to S0.

The simulation outputs matched the expected values for all states.

11. FUTURE IMPROVEMENTS

The current design can be enhanced by incorporating additional features to improve functionality and real-world applicability. Possible improvements include:

- Adding timer-based traffic signal control for realistic operation.
- Implementing pedestrian crossing support.
- Introducing emergency vehicle priority handling.
- Expanding the design to support multi-road intersections.
- Developing an adaptive traffic control system based on traffic density.

12. CONCLUSION

The Traffic Light Controller was successfully designed and verified using **Verilog HDL** and a **Moore Finite State Machine (FSM)** architecture. The controller correctly performed all state transitions and generated the expected traffic light outputs based on the sensor inputs. Simulation results in **Xilinx Vivado** confirmed the correct functionality of the design. This project strengthened practical skills in **RTL Design, FSM Design, Verilog HDL, and Functional Verification**, providing a solid foundation for more advanced digital design projects.

13. REFERENCES

1. M. Morris Mano and Michael D. Ciletti, *Digital Design*, 6th Edition, Pearson Education, 2018.
2. David Money Harris and Sarah L. Harris, *Digital Design and Computer Architecture*, 4th Edition,
3. Xilinx Inc., *Vivado Design Suite User Guide*, AMD/Xilinx Documentation.
4. [OpenAI ChatGPT](#) – Used for guidance on Verilog coding, documentation structure, debugging, and technical explanations.
5. [Anthropic Claude](#) – Used for reviewing technical documentation, improving report writing, and refining explanations.